TECHNICAL MANUAL

TeslesSeasons

Real-calendar seasons for Minecraft 26.2, with a season-neutral Voxy LOD integration

---

Version 1.0.0 · Minecraft 26.2 · Fabric
Integrations Voxy 0.2.18-beta · VoxyServer 1.2.4 · Dynamic Trees 1.8.0 · Iris · Sodium
Verified by 93 contract tests and two build-time wiring verifiers
Scope Architecture, season model, world mutation, Voxy integration, extension points, configuration, diagnostics

Contents

What is in this manual

Part 1

The shape of the mod

TeslesSeasons drives a Minecraft world through a real four-season year: leaves colour and fall, flowers and mushrooms come and go, snow accumulates and melts, and terrain far beyond the player's render distance shows the same season as the ground under their feet. This part explains the single idea the whole mod is built on, and the rules that follow from it.

The one-authority model

Every seasonal mod eventually faces the same question: when three different parts of the game need to know what season it is, who decides? The answer here is that exactly one thing interprets the calendar, and everything else is a projector of what that thing decided.

```
  RealCalendarSeasonClock  /  SeasonDebugController      the clock
             |
             v
     SeasonRegistry            one SeasonSector per season, replaceable
             |
             v
     SeasonDirector            mints a revision only when targets change
             |
             v
  ==== immutable SeasonFrame ====   absolute targets, never deltas
             |
    +---------+-----------+-----------+--------------+
    |         |           |           |              |
  Snow     Leaves      Flora     World effects    Voxy LOD
  System   System      System     (pluggable)     projection + shader
```

The **SeasonDirector** is the only code in the mod that reads a clock and concludes "it is autumn". It hands out an immutable `SeasonFrame`: a record of about twenty numbers describing what the world should look like right now. Nothing downstream is allowed to re-derive a season, cache one, or remember what it was doing last tick.

This sounds like ceremony until you see what happens without it. If the snow code decides the season from the clock, and the LOD renderer decides it from its own copy of the clock, the two answer differently the instant one of them runs a few milliseconds later than the other — and the seam between near terrain and distant terrain is exactly where a player is looking.

The nine rules

These are not style preferences. Each is written down because breaking it produced a specific, diagnosable failure in a running world.

RULE	WHAT IT MEANS, AND WHAT BREAKING IT LOOKS LIKE
One season truth	Only `SeasonDirector` interprets the calendar. Nothing else derives, caches or reinterprets a season. Break it and near and far terrain disagree at the seam.
Percent means world, not progress	A target of 25 % means a quarter of the world, chosen by a deterministic coordinate field — never "25 % of the work queue is done". Break it and the same coordinate is snowy or bare depending on which chunk the sweep reached first.
One coordinate field	`SeasonCoordinateField` is thresholded identically by the physical projector, the LOD mesh projector and the GLSL shader. This is what lets near and distant terrain agree without storing any season history anywhere.
Targets are absolute, never deltas	A frame says what the world should be, not how to get there. A player who logs in mid-winter must see the same world as one who watched it arrive.
Voxy persistence is season-neutral, and heals	Season is applied when a section is meshed, never when it is stored. Suppression is directional: writes that move the world away from neutral are held out of the LOD store; writes that bring it back are deliberately let through.
Ownership before deletion	Only blocks recorded in a chunk's ledger are ever removed. Player-placed snow, ice and plants are never adopted and never deleted.
Identity decides category, not superclass	What a block is comes from its registry id. A wild berry bush is berry flora whether it extends a bush block or a crop block.
Absent means absent	A leaf the frame removes is AIR, never an invisible collision box and never a fragment-shader discard — a discard is invisible to the shadow pass, so discarded leaves keep shading the ground.
Grass identity is never changed	Winter comes from snow layers and tint, never from swapping `grass_block` for something else. That swap destroys the block a player sees and cannot be reversed faithfully.

> **The shortest summary of the whole design.** One object describes the world's current appearance. One pure function turns a coordinate into a yes/no for any percentage. Everything else reads those two and converges what it owns.

Where each rule came from

The mod's history is a useful map of where the difficulty actually lies, and every entry below was found in a running world rather than by reading code.

FAILURE	ROOT CAUSE
Whole subsystem silently inert	Fifteen mixins that no config referenced, and a `ClientModInitializer` that was not an entrypoint. Invisible by reading; both now fail the build.
Snow flickering between near and far	The LOD projector used a multi-octave "organic" noise field while the server used the flat coordinate hash.
Distant terrain kept last winter's snow forever	The projector could add snow but never remove it.
Only the player's surroundings updated	The column sweep was keyed on the season revision and reset whenever it changed — which, in a fast timelapse, was more often than any chunk came up for its turn.
Snow in the treetops, shadows under bare trees	The projector took the topmost occupied voxel as the ground. In a forest that is the canopy.
Lakes rendered as flat white sheets	The shader gave every unclassified up-face a snow cap, and water is unclassified.
Distant trees never regained leaves	LOD suppression was blind to direction, so the spring restoration that would have healed the store was blocked by the same flag that blocked the autumn removal.
Eight of nine berry blocks not seasonal	They extend a crop block, and the crop guard ran before the berry rule.
Thaw punched holes in the snowfield	Footprint and depth retreated together, so a column under four layers dropped to bare ground the moment the edge passed it.

Part 2

The season model

This part is the contract. It defines the calendar, the twenty channels a frame carries, and exactly what each of the twelve season-phases asks the world for. Everything else in the mod is a consequence of these numbers.

Calendar, seasons and phases

The year runs **Summer → Autumn → Winter → Spring → Summer**. Each season has three phases, giving twelve in all:

PHASE	ROLE
INCOMING	The season arriving. Takes over exactly where the previous season's Outgoing left off.
STABLE	The season itself. The long middle where most of the visible change happens.
OUTGOING	The season handing over. Ends at exactly the value the next season's Incoming begins at.

Incoming and Outgoing are each seven real days by default (`incomingTransitionDays`, `outgoingTransitionDays`); Stable fills the rest of the season. The clock is real-calendar and refreshes every `calendarRefreshSeconds` (30 by default), in the timezone given by `timeZone`.

Every phase exposes a single **progress** value `p` in [0, 1]. Every formula in this part is a function of the phase and that one number — nothing else. That is what makes a season module a pure function and lets the contract suites evaluate all twelve phases without a running game.

The SeasonFrame

A `SeasonFrame` is an immutable record of what the world should look like now. Each field is an **absolute world target** in [0, 1] — a retention of 0.6 means "sixty percent of these should exist", not "we are sixty percent of the way through removing them".

CHANNEL	WHAT IT TARGETS
`season, phase, progress`	Where the calendar is. Inputs, not targets.
`revision`	Monotonic. The only non-target field; exists purely so schedulers can coalesce.
`autumnColor`	How far the foliage palette has turned.
`leafRetention`	Fraction of deciduous leaves that still exist, physically.
`flowerRetention`	Fraction of flowers that still exist.
`plantRetention`	Fraction of ground vegetation that still exists.
`mushroomRetention`	Fraction of mushrooms that still exist.
`berryRetention`	Fraction of wild berry bushes that still exist.

CHANNEL	WHAT IT TARGETS
`groundDormancy`	How brown and dormant the ground reads.
`groundFrost`	How frozen the ground is. Drives the pluggable ice effect.
`snowCoverage`	Fraction of eligible columns that carry snow.
`snowDepth`	Normalised depth; ×8 gives the layer target.
`springFreshness`	How vivid the new growth reads.
`treeGrowthFactor`	Handed to Dynamic Trees.
`seedDropFactor`	Handed to Dynamic Trees.
`fruitProductionFactor`	Handed to Dynamic Trees and food mods.
`snowAccumulating / snowThawing`	Direction flags. Thawing switches on the layer-by-layer melt.

Quantisation, and why it matters more than it looks

Targets are compared at a granularity of **1/256**. This is the single most performance-critical constant in the mod. A revision bump means "redo the world": the server re-queues every loaded chunk and the client invalidates every Voxy LOD section. Comparing raw floats makes that happen on every clock refresh, because the calendar advances continuously and any two samples thirty seconds apart differ in the last few bits.

One step per 1/256 over a seven-day phase is roughly one step every forty minutes — far below what a player notices, and far above the rate at which rebuilding the LOD set is affordable. `progress` is deliberately excluded from the comparison: it is the clock input, not a target. If every derived target is unchanged, the world has nothing to do however much raw progress has advanced.

What each season asks for

In the tables below, `p` is phase progress. These are the complete formulas.

Summer

CHANNEL	INCOMING	STABLE	OUTGOING
`leaves / flowers / plants / berries`	`1.0`	`1.0`	`1.0`
`mushrooms`	`0.0`	`0.0`	`0.0`
`autumnColor`	`0.0`	`0.0`	`p`
`seedDrop`	`0.0`	`0.0`	`p`
`growth / fruit`	`1.0`	`1.0`	`1.0`
`dormancy / frost / snow`	`0.0`	`0.0`	`0.0`

Summer is the neutral world. Everything exists, nothing is frozen. Its Outgoing phase does one job: turn the palette so Autumn Incoming can start at a fully saturated 1.0 with no jump.

Autumn

CHANNEL	INCOMING	STABLE	OUTGOING
autumnColor	1.0	1.0	1.0
leaves	1.0	1 − p	0.0
flowers / berries	1.0	1 − p	0.0
plants	1.0	1.0	1 − p
mushrooms	p	1.0	1 − p
dormancy	p	1.0	1.0
frost	0.15·p	0.15 + 0.25·p	0.40 + 0.60·p
snow coverage	0.0	0.0	p
snow depth	0.0	0.0	0.125 (1/8)

Two details are load-bearing. Mushrooms are an autumn-only crop: they rise through Incoming, hold through Stable and are gone by the end of Outgoing. And Autumn Outgoing lays down the **first snow footprint** at exactly one eighth of a layer, growing to full coverage — so Winter Incoming inherits a complete 1/8 blanket rather than starting from nothing.

Winter

CHANNEL	INCOMING	STABLE	OUTGOING
leaves / flowers / plants / mushrooms / berries	0.0	0.0	0.0
autumnColor	1 − p	0.0	0.0
dormancy / frost	1.0	1.0	1.0
snow coverage	1.0	1.0	1 − p
snow depth	0.125	max(0.125, p)	1 − p
accumulating / thawing	accumulating	accumulating	thawing

Winter Incoming is a seamless handoff: it takes the complete 1/8 footprint Autumn Outgoing finished building and must never reset coverage to zero. Winter Stable maps progress directly to depth — at 54 % the target is 4.32 layers, realised as a deterministic mix of 4/8 and 5/8 placements so that the large-area mean converges to 4.32. Winter Outgoing brings footprint and depth down together; how an individual column gets from its depth to zero is covered in Part 4.

Spring

CHANNEL	INCOMING	STABLE	OUTGOING
flowers / plants / berries / growth / fruit	p/3	(1+p)/3	(2+p)/3
leaves	0.0	p	1.0
dormancy / frost	1 − p	0.0	0.0
freshness	p	1.0	1 − p
mushrooms / snow	0.0	0.0	0.0

Spring's flora restoration is one continuous 0→1 channel spanning all three phases, each covering a third, so Summer starts at exactly 100 %. Leaves return across Spring Stable and then stay full.

> **Reading the tables as a player.** Autumn Stable is when the canopy actually comes down. Winter Stable is when snow gets deep. Spring Stable is when the leaves come back. The Incoming and Outgoing phases are almost entirely about handing over cleanly.

Continuity

Every continuous channel must leave a phase at exactly the value the next phase enters with, at all twelve boundaries. Not approximately — exactly. `SeasonContinuityTest` checks fourteen channels across twelve boundaries, 168 comparisons, and a run of a full simulated year in a live server confirmed all twelve boundaries continuous in practice.

This is why several formulas look more elaborate than they need to. Winter Incoming fades `autumnColor` from 1 to 0 rather than snapping it, because Autumn Outgoing ends fully saturated. Winter Stable's depth is `max(0.125, p)` rather than `p`, so that at p = 0 it still equals the 1/8 endpoint Winter Incoming handed it.

Revisions

The director owns a monotonic revision counter, and mints a new revision **only** when a frame's targets actually differ from the one in place. A Stable phase that asks for nothing new never triggers a world pass. When targets are unchanged the frame is still refreshed — so progress keeps advancing for anything reading it — but the old revision is carried forward.

Downstream, a revision is a coalescing token. If revisions 53, 54 and 55 all arrive before a chunk is processed, the chunk is taken straight to 55; the intermediate states are never realised.

Part 3

The coordinate field

One pure function turns a percentage into a concrete set of world coordinates. It is the smallest piece of the mod and the one everything else depends on.

Why percentages must mean coordinates

Suppose the frame says snow coverage is 25 %. Something has to decide which quarter of the world is snowy. There are two ways to do it, and only one of them works.

The tempting way is to keep a record: mark chunks as done, snow a quarter of them, remember which. This fails immediately at scale. The server can only reason about loaded chunks, and the client's LOD renderer is drawing terrain kilometres away that no server tick has touched in hours. Any record-keeping scheme means near and far are working from different books.

The way that works is to make membership a **pure function of position**. Given a coordinate and a world seed, a hash produces a value in [0, 1). A coordinate belongs to the target set at retention `p` exactly when its value is below `p`:

```
existsAtRetention(fieldValue, retention)  ⇔  fieldValue < retention
```

Three consequences follow, and they are the reason the whole design holds together:

- **No history is stored anywhere.** Any consumer can answer "is this coordinate snowy at 25 %" without knowing anything about what happened before.
- **Near and far agree by construction.** The physical block projector, the LOD mesh projector and the GLSL shader threshold the same function, so they cannot disagree.
- **Coverage is monotone.** As retention rises from 0 to 1, the selected set only ever grows. Nothing pops in and out as the season advances.

Salts

Each channel gets its own constant mixed into the hash. Without that, a column that is snowy would also be more likely to keep its flowers — and correlated channels produce visible stripes where two things change together.

SALT	VALUE	CHANNEL
`SNOW_COVERAGE_SALT`	0x6A09E667	Which columns are snowy
`SNOW_DEPTH_SALT`	0xBB67AE85	Fractional-layer dithering
`LEAF_SALT`	0x3C6EF372	Which leaves survive
`FLOWER_SALT`	0xA54FF53A	Which flowers survive
`GROUND_PLANT_SALT`	0x510E527F	Which ground plants survive
`MUSHROOM_SALT`	0x9B05688C	Which mushrooms exist
`BERRY_SALT`	0x1F83D9AB	Which wild berry bushes survive
`ICE_SALT`	0x5BE0CD19	Reserved for the built-in freezing effect

The values are the fractional bits of square roots — arbitrary but well separated. `SeasonFieldStatisticsTest` checks that the field is uniform, that it is pure, and that the per-channel salts are genuinely independent of one another.

> **Adding an effect?** Give it a salt nobody else uses. Two effects sharing a salt select the same coordinates, and the correlation shows up in game as the two appearing and vanishing in exactly the same patches. Use `SeasonCoordinateField.effect01(x, z, seed, yourSalt)`.

Snow uses two dimensions, flora uses three

Snow coverage and depth are functions of `(x, z)`: snow is a property of a column, and every block in a column shares its answer. Leaves and flora hash `(x, y, z)`, because a tree's canopy has many leaves in one column and they should not all vanish together.

That extra dimension creates one hazard. A double-height plant occupies two positions and would get two different answers, leaving a headless lower half or a floating upper one. Both halves therefore resolve their membership at the lower half's coordinate — see `decisionPos` — and removal is all-or-nothing: a double-height removal is not started unless there is budget to finish it. `TallPlantAtomicityTest` holds this.

The GLSL mirror

The distant-terrain shader needs the same answers, so `snowCoverage01` is mirrored bit-for-bit in GLSL by the Voxy shader patch. Both sides run the same avalanche:

```
h  = x·0x9E3779B9 ^ z·0x85EBCA6B ^ seed ^ salt
h ^= h >>> 16;  h *= 0x7FEB352D
h ^= h >>> 15;  h *= 0x846CA68B
h ^= h >>> 16
value = (h & 0xFFFFFF) / 16777215.0
```

> **Do not "improve" the snow hash.** Changing the mixing constants on the Java side without changing the shader breaks near/far parity in a way that only shows up visually, kilometres from the player, in one season. `VoxySnowParityTest` guards the Java side of this contract, and `VoxyShaderPipelineTest` runs the real injection chain against Voxy's actual shader source.

Dimension separation is carried by the per-world visual seed rather than an extra hash term, which keeps the GLSL mirror a straight function of `(x, z, seed)`.

Quantised targets

Selection reads `snowCoverageTarget()` and `snowDepthTarget()` rather than the raw channels. Both are quantised to 1/256. The reason is subtle but important: the world is mutated once per revision, so if selection read the raw channel, the physical blocks placed at the start of a revision would already disagree with what the LOD projector computes later in the same revision — drifting slightly out of parity between every update, forever. Quantising at the point of selection makes "same revision" mean "identical targets" by construction.

Part 4

The physical world

How real blocks are brought to the frame's targets: the sweep that visits them, the ledgers that make removal reversible, and the budget that keeps it all inside a server tick.

The reconciler

`SeasonalWorldReconciler` holds a `WorkState` per loaded chunk and walks them continuously, converging each toward the current frame. It never asks "what changed?" — it asks "what should this column look like, and what does it look like?" That difference is the whole of its job, and it is why a chunk that has been unloaded for a year needs no catch-up logic.

Each chunk carries two independent sweeps:

SWEEP	WHAT IT DOES PER COLUMN
Surface	Snow placement and removal, flora removal and restoration, then any registered world effects.
Canopy	Walks down from the motion-blocking height through up to `canopyScanDepth` blocks, removing and restoring deciduous leaves and canopy mushrooms.

The column sweep

A chunk has 256 columns. A sweep visits each exactly once and then wraps. The order within a sweep is a permutation:

```
seed  = chunkX·73428767 ^ chunkZ·912931 ^ sweepNumber
start = seed & 0xFF
step  = (seed >>> 8 | 1) & 0xFF        // forced odd
column(i) = (start + i·step) & 0xFF
```

An odd step modulo 256 is a full-cycle permutation, so one sweep touches all 256 columns exactly once and no column can starve. The `| 1` is what forces it odd; an even step would revisit a fraction of the columns and never reach the rest.

> **The cursor is deliberately not reset when the season changes.** It used to be, and that quietly starved everything outside the player's vicinity. A surface revision is quantised at 1/100 per channel, so during a fast timelapse it changed roughly twice a second while any individual chunk came up for its turn only every couple of seconds. The cursor was back at zero on nearly every visit, the same opening columns were redone forever, and the rest of the chunk was never reached. Near chunks looked right only because the player hotset canonicalises them in full by a different path.

Whether a chunk is current is tracked separately, by how many columns it has covered since the revision last changed. Once that reaches 256, the sweep has necessarily covered every column at least once under this revision or a newer one.

Ownership and ledgers

Seasonal change has to be reversible, and it has to be reversible without touching anything that was not seasonal. Each chunk therefore carries persistent ledgers, stored as Fabric attachments:

LEDGER	HOLDS
`owned_snow_v3`	Packed positions of snow this mod placed
`removed_leaves_v3`	Encoded block states of leaves removed, and where
`removed_flora_v4`	Encoded block states of flora removed, and where
`effect_owned_v1`	Positions owned by pluggable world effects, keyed by effect id

> **The ownership rule.** Only snow recorded in the owned-snow ledger is ever removed. A player-placed snow layer that happens to sit where the season also wants snow stays the player's: it is never overwritten, and never deleted in the melt. The same holds for every pluggable effect, which gets its own ownership set automatically.

The ledgers are flushed on every dirty work slice rather than at the end of a chunk's pass. Writing the block and writing its ledger entry must not be separable: a crash between the two would leave the world with the leaf gone and no record of what to restore. A few extra metadata writes per tick are much cheaper than that.

Snow

The target for a column is:

```
coverage = frame.snowCoverageTarget()        // quantised
depth    = frame.snowDepthTarget()           // quantised
if coverage ≤ 0 or depth ≤ 0                     → 0 layers
if snowCoverage01(x, z, seed) ≥ coverage         → 0 layers

target = depth · 8
layers = floor(target) + (snowDepth01(x,z,seed) < frac(target) ? 1 : 0)
```

The fractional part is realised by **deterministic stochastic rounding**: at a target of 4.32 layers, only 4/8 and 5/8 states are produced, and the proportion of 5s is chosen so the large-area mean converges to 4.32. No column ever holds an illegal layer count.

Where snow may rest

Snow settles on whatever solid surface is there — stone, gravel, a roof, a fallen log — matching what vanilla's own placement allows. It is refused on water and lava, on ice, and on foliage. The projector has no `Level` at mesh-build time, so the rule is expressed over a block state alone, in `SnowSystem.canRestOn`, and both the server and the LOD projector ask it. Blocking motion stands in for a sturdy top face.

Melting, a layer at a time

The canonical mapping brings footprint and depth down together. Applied literally per column that means a column standing under four layers drops to bare ground the instant the

retreating footprint passes it — which reads as holes being punched through full-depth snow with grass at the bottom of them.

The specification pins the aggregate, not the trajectory, so each layer is instead given its own slice of retreating coverage:

```
if thawing:
    target = min(depth·8, (coverage − fieldValue) / 0.02)
layers = max(1, …)          // a column inside the footprint keeps its last layer
```

A column therefore comes down 4/8, 3/8, 2/8, 1/8, ground, and the melt window is proportional to how deep the snow is. The band is at least the revision quantum wide, so no column can lose more than half a layer between two updates. Because a column inside the footprint always keeps its last layer, the footprint matches the specification exactly rather than approximately; mean depth sits slightly under nominal (about 1.9/8 where the specification says approximately 2/8), which is the cost of removing the cliff.

What appears when the last layer goes

Spring restores real flora gradually, tracking the plant channel — at 45 %, only 45 % of a column's canonical flora is back. A placeholder covers the remainder so the ground never reads as bare dirt. Ground cover is an ordered list of candidates: `minecraft:short_grass` first, then the slab-mounted equivalent for slab terrain, then nothing. A single-candidate rule silently degraded to air on slab ground, leaving those columns visibly bare all spring while ordinary ground beside them was covered.

Leaves

A deciduous leaf exists when `leaf01(x,y,z,seed) < leafRetention`. Removal writes the exact block state into the ledger; restoration reads it back and puts the same state, same species, in the same place.

> **Absent means absent.** A removed leaf becomes AIR. It is not made invisible, and it is not discarded in the fragment shader — a discard only applies in the pass running the patched shader, and Iris renders its shadow map with a different program, so discarded leaves go on casting full-canopy shadows over bare winter trees.

Evergreens are classified separately and keep their needles all year. Dynamic Trees ships several leaf block subclasses, not just one, and all of them are classified centrally — matching only the exact class left every subclass unclassified and those leaves survived winter as floating foliage.

Flora

Flora is classified once, at startup, from the real block registry into an identity table. Keyword heuristics run over that registry a single time to build the table and never again; they are not the production lookup. The result is logged, which is how a misclassification becomes visible:

```
Seasonal flora registry resolved from 1596 installed blocks:
148 plants, 77 flowers, 50 mushrooms, 9 berries.
```

Five categories exist: `PLANT`, `FLOWER`, `MUSHROOM`, `BERRY` and `NONE`. Each tracks its own frame channel and its own field salt.

> **Identity decides category, not superclass.** Wild berry bushes are resolved before every other rule, including the crop guard. Some ship as subclasses of a crop block, and an `instanceof CropBlock` test reaches them first and files them as not-seasonal — which once left the berry channel holding exactly one block out of nine. A wild bush is worldgen flora whatever it extends; only cultivated crops belong to a farming mod.

Budget

All of this runs on the server thread, so it is bounded twice over: by a write count and by a wall-clock deadline. Both are checked by a single `canWork()`, and every loop that could be long checks it.

BOUND	DEFAULT
`worldReconcileBudgetMicros`	2200 µs per tick
`debugReconcileBudgetMicros`	4500 µs per tick (while a timelapse runs)
`surfaceColumnsPerTick`	1024, clamped to 96 outside debug
`leafColumnsPerTick`	128
`maxVisibleSnowWritesPerTick`	128
`maxVisibleLeafWritesPerTick`	24

Two paths are deliberately unbudgeted, because correctness beats smoothness there: the player hotset, and the pre-send pass that canonicalises a chunk before it is transmitted. A chunk must never become visible carrying a season the director has already moved past.

That pre-send pass is also the most expensive thing the mod does when a player is flying, so it short-circuits: when the frame takes nothing out of the world and the chunk owes nothing back, the guarantee holds trivially and both passes are skipped entirely. For most of the year that is every chunk.

Part 5

Voxy integration

Making terrain kilometres away show the same season as the ground underfoot, without ever storing a season in the LOD database. This is the hardest part of the mod and the source of most of its history.

Why LOD is hard here

Voxy keeps its own persistent database of the world at several levels of detail, built as regions are played and streamed to clients. It is a cache of what the world looks like — and that is exactly the problem, because what the world looks like changes four times a year.

Three approaches are possible, and two of them fail:

APPROACH	WHY IT FAILS, OR DOES NOT
Store the season in the LOD	The store is only written where a player has been. A region last visited in winter stays winter for good, and the player sees a band of the wrong season wherever they happened to be standing months ago.
Rebuild the LOD when the season changes	The server can only rebuild what is loaded, and the LOD covers far more than that. It also means rebuilding the entire visible LOD set on every revision.
Keep the store season-neutral and apply the season at mesh time	The store holds one world — full canopy, no snow. The season is painted on as sections are meshed, so a section rebuilt today gets today's season and a section stored last winter carries no winter at all.

The third is what this mod does, and every rule below exists to keep the store neutral.

Season-neutral persistence

Neutrality is maintained in three places, because there are three ways world data reaches Voxy.

1 · Chunk ingest

When Voxy ingests a chunk, it is handed a neutralised copy rather than the live one. The copy is built from the chunk's ledgers: seasonal snow is taken back out, removed leaves and flora are put back. A chunk that is physically deep in winter is therefore handed over looking like summer.

This runs on the server as well as the client, because VoxyServer's own voxelizer funnels through the same Voxy entry point. Chunk load, chunk unload and re-voxelisation after a block change all pass through it; there is no section-level bypass.

2 · Live block changes

A seasonal block change would ordinarily mark the chunk dirty and cause a re-voxelisation. Whether that is allowed depends on **which direction the change moves the world**.

> **Directional suppression — the rule that took longest to find.** Writes that take the world away from neutral are held out of the LOD store. Writes that bring it back are deliberately let through.

CHANGE	DIRECTION	REACHES THE LOD?
Leaf removed	away	no
Leaf restored	toward	**yes**
Snow placed, or deepened	away	no
Snow thinned, or melted	toward	**yes**
Flora removed	away	no
Flora restored	toward	**yes**

Blocking both directions looks more conservative and is in fact the opposite: it freezes each region's LOD at whatever season was running the first time that region's LOD was built. A forest whose LOD was born in winter then had no canopy and no way to gain one, because the spring restoration that would have healed it was suppressed by the same flag that had suppressed the autumn removal — and the client cannot invent a canopy the store does not contain. Letting the healing direction through makes the store self-correcting: over one year every leaf is restored, every flower returns and every layer melts, and each of those writes reaches the LOD.

The direction is read from the blocks themselves, not from which helper the caller reached for — the same code path places snow and melts it, drops a leaf and restores it.

3 · Bulk import

Rebuilding the store from region files on disk is the one operation that can neutralise an entire world at once, and it is hooked the same way: as each chunk's sections are decoded, the persisted ledgers are applied to them before Voxy sees them. This is the supported remedy for a store that was contaminated by an older build.

Mesh projection

With the store neutral, the season is applied when a section is meshed. The projector takes a copy of the raw section data on its way into the mesher and edits it.

```
for each section handed to the mesher:
   1. stamp it with the geometry key it is being meshed against
   2. strip leaves the frame has dropped          (all LOD levels)
   3. for each of the 32×32 columns:
        a. clear seasonal snow at every height
        b. if the frame wants snow here, place it on the surface
   4. hand the edited copy to the mesher; the store is untouched
```

The projection pass, in order. Steps 2 and 3a are removals and run unconditionally; 3b is the only addition.

Leaves come off first. Snow is placed on the surface of a column, so a canopy about to be deleted must not be allowed to act as that surface. Doing snow first left a lid of snow hanging in the air exactly where the winter canopy had been — and that floating lid went on shading the ground beneath it, which is what "shadows from leaves that are not there" actually was.

The surface search ignores foliage. The server locates a column's surface with the motion-blocking-no-leaves heightmap, which looks straight through a canopy. A search that stops at the first non-air voxel finds the treetop in every forested column and treats it as ground — putting snow up in the trees.

Removal happens at every height, and before any decision. Checking only the topmost voxel left snow stranded wherever something stood above it: under a trunk, or under a canopy at coarse LOD where tree and ground merge into one column. Clearing before deciding matters too — a column still holding last winter's snow at its surface offers no ground for this winter's snow, so its depth could never be corrected either.

Never punch a hole. Clearing snow at coarse LOD replaces the voxel with the material below it. When there is nothing below — which is the case for the bottom row of every section — the voxel is left alone. Falling back to air punched holes through the landscape in bands. Stale snow is a far smaller fault than terrain you can see the sky through.

LOD LEVEL	WHAT THE PROJECTOR DOES
0	One voxel is one block. Exact coordinate-for-coordinate parity with the server.
1 – 2	Layer counts scaled to the voxel's resolution; a thin dusting at level 2 is dropped rather than rendered as a full voxel of snow.
3 and above	No snow geometry — a voxel spans too many blocks for per-column snow to mean anything. Appearance comes from the shader. Leaf stripping still applies at every level.

Geometry keys and cache bypass

Colour reaches Voxy as shader uniforms uploaded per bind, so it is deliberately not part of what decides whether a section must be rebuilt — including it would rebuild the whole LOD set every time the calendar moved at all. Only two things change LOD geometry: seasonal snow, and leaves the frame has dropped. Those form a **geometry key**. Each section is stamped with the key it was meshed against, and a cached mesh built under an older key is refused.

The shader pipeline

Voxy's own vertex and fragment shaders are patched by string injection at load time. Three layers apply in order, and each is idempotent — patching an already-patched source is a no-op, which matters because the render pipeline is recreated several times during startup.

LAYER	WHAT IT ADDS
Vertex season bridge	Carries a 4-bit seasonal category and the fragment's world position through to the fragment stage, packed Iris-safely into the custom id.
Fragment snowfield bridge	The GLSL mirror of the coordinate field, distance blending, per-category seasonal colour, terrain-side snow shading, and discards for flora the season has removed.
Canonical visual post-pass	Luminance-preserving autumn foliage shaping, so distant leaves match the near field rather than washing out under shaders.

Twelve uniforms are bound, by exactly one binder. Two separate mixins used to inject into the shader bind method, each resolving and writing its own subset; whichever ran last won, and the two disagreed about where their values came from.

> **Nothing gets a blanket snow cap.** An earlier version gave every fragment in the unclassified category a snow cap on its up-face, guarded by a preprocessor check meant to spare water. Voxy draws water through the same program, so at full winter every lake and river became one flat white plane at its own water level. LOD 0–2 now receives real projected snow geometry on any solid surface, classified or not, so the cap was not what carried distant snow anyway.

Handoff

Near the player, terrain is drawn from real blocks and Voxy's copy is hidden behind them. Sections in that band used to have snow stripped but never added, on the theory that adding season near real blocks would show it twice at the seam. It does not — both sides threshold the same field at the same target and agree column for column. What the asymmetry produced was a band of LOD that had been cleared of snow and never given any back: a flat, empty strip that followed the player around all winter. Snow is now projected symmetrically at every section within the projected LOD range.

What the client cannot do

> **The projector removes; it cannot invent.** It can take away leaves the season has dropped and snow the season does not want. It has no way to grow leaves that the store does not contain. This is the one asymmetry that matters operationally: if a region's LOD was written while that region was bare, only a rebuild will fix it — see Part 8.

Part 6

Extending it

Two things are pluggable: what a season is, and what a season does to the world. This part is a how-to.

Replacing a season

A season is a pure function from a clock snapshot to a frame of absolute targets. Write one, register it, and nothing else in the mod changes:

```
public final class HarshWinterSector implements SeasonSector {
   public SeasonFrame frame(SeasonSnapshot raw) {
      float p = SeasonSector.clamp(raw.phaseProgress());
      return SeasonFrame.builder(raw.season(), raw.phase(), p)
         .leaves(0.0F).flowers(0.0F).plants(0.0F)
         .dormancy(1.0F).frost(1.0F)
         .snow(1.0F, Math.max(0.25F, p))    // deeper, sooner
         .thawing(raw.phase() == CalendarPhase.OUTGOING)
         .build();
   }
}

SeasonRegistry.register(Season.WINTER, new HarshWinterSector());
```

`SeasonDirector` reads the registry rather than naming the four built-ins, so there is no switch to edit and no other file to touch.

> **A replacement is held to the same contract.** The checkpoint and continuity suites resolve through the registry, so they run against whatever is registered. A replacement season that breaks phase continuity fails the build, not the world.

What a season module must not do

- Touch the world, the scheduler, or another mod. It is a formula, nothing else.
- Remember anything between calls. Same snapshot in, same frame out, forever.
- Leave a phase at a different value than the next phase enters with.

World effects

Snow, leaf fall and flora are built into the reconciler because the specification pins them. Everything else a season might do to the world — lakes freezing, puddles, frost on glass, ice thickening, mud in spring — is a `SeasonalWorldEffect`: one class, one registration.

```
public interface SeasonalWorldEffect {
   String  id();                                        // ledger key; treat it like a block id
   boolean appliesTo(SeasonFrame frame);                // cheap gate, asked once per chunk
pass
   boolean applyToColumn(SeasonalEffectContext ctx);   // false = out of budget, retry later
}
```

The context, and why to use it

YOU CALL	YOU GET
`ctx.frame()`	Absolute targets. Never ask how far through a transition you are.
`ctx.seed()` + `effect01(…, yourSalt)`	The same deterministic field the snow and the distant LOD use.
`ctx.surface()`	The ground, ignoring foliage — the same position the snow pass calls the surface.
`ctx.place(pos, state)`	A budgeted write, recorded as **yours**.
`ctx.restore(pos, state)`	Undoes one of your placements; silently declines anything else.
`ctx.ownedInColumn()`	What you own here, so you can take it back.
`ctx.canWork()`	False once the tick's budget is spent.

Three properties you get free

PROPERTY	WHY IT HOLDS WITHOUT YOU DOING ANYTHING
Player builds are safe	`restore` only touches what `place` recorded, against a ledger keyed by your effect id. A thaw cannot eat ice a player put there.
Distant terrain agrees	`place` is a divergence from the neutral world by definition and is held out of Voxy's LOD store; `restore` is a healing write and is let through. The store keeps converging on neutral without your effect knowing that store exists.
The tick survives	Every write is budgeted. Return `false` when one does and the column is picked up again where it left off.

Registration order is the run order, and registering the same id twice replaces the earlier one — which is how a pack overrides a built-in effect rather than fighting it. An effect that throws is logged once and skipped for that column; it cannot take the season system down with it.

Worked example: lake ice

`WaterFreezeEffect` ships as both a working feature and the worked example. It is the whole of winter lake ice, and nothing in the reconciler, the season modules or the Voxy path knows it exists.

```
public boolean applyToColumn(SeasonalEffectContext ctx) {
   BlockPos surface = ctx.surface();
   float coverage   = ctx.frame().groundFrost() * MAXIMUM_COVERAGE;
   boolean wantIce  = coverage > 0.0F
      && effect01(ctx.x(), ctx.z(), ctx.seed(), ICE_SALT) < coverage;

   // give back what is no longer wanted first
   for (BlockPos owned : ctx.ownedInColumn()) {
      if (!ctx.canWork()) return false;
      boolean keep = wantIce && owned.equals(surface);
      if (!keep && ctx.stateAt(owned).is(Blocks.ICE)
              && !ctx.restore(owned, sourceWater())) return false;
   }
   if (!wantIce) return true;

   BlockState state = ctx.stateAt(surface);
   if (!isFreezableWater(state) || !ctx.stateAt(surface.above()).isAir()) return true;
   return ctx.place(surface, Blocks.ICE.defaultBlockState());
}
```

Worth reading for what it does not do. It keeps no state between ticks — the frame says how much of the world should be frozen and the field says which part, so a player who logs in mid-winter sees the same lake as one who watched it freeze. It never removes ice it did not place. And it never asks how far through winter we are, only how frozen the world should be now.

It is **off by default**: it is not part of the season contract, and turning it on changes what an existing world looks like. Enable it with `seasonalWaterFreezing`.

Ideas that fit this seam cleanly

EFFECT	CHANNEL TO KEY ON	SKETCH
Frozen crops	`groundFrost`	Swap mature crops to a withered variant while frost is high; restore on thaw.
Spring mud	`springFreshness`	Place mud on a fraction of dirt columns as the thaw runs.
Autumn leaf litter	`1 − leafRetention`	Place litter under deciduous canopy as the canopy comes down.
Icicles	`groundFrost`	Hang from overhangs; needs a downward search rather than `surface()`.
Thicker ice over winter	`snowDepth`	Replace ice with packed ice past a depth threshold — a second effect layered on the first.

Give each effect its own salt. Two sharing one select the same coordinates, and the correlation shows up in game as the two appearing and vanishing in exactly the same patches.

Part 7

Configuration reference

All 52 options in `config/teslesseasons.json`, written on first run. Cosmetic options are yours to set; the values the season contract depends on are re-pinned on every load.

> **Re-pinned values.** `TeslesSeasonsConfig.enforceCanonicalTargets()` resets the options the specification fixes every time the config is loaded — snow footprint, layer count, the transition fractions. Editing those in the file has no effect, on purpose: they are contract, not preference, and a world running with them changed is not running the season model this manual describes.

OPTION	DEFAULT	NOTES
CALENDAR		
`timeZone`	`"Europe/Helsinki"`	Timezone the real calendar is read in.
`incomingTransitionDays`	`7`	Length of every Incoming phase, in real days.
`outgoingTransitionDays`	`7`	Length of every Outgoing phase, in real days.
`calendarRefreshSeconds`	`30`	How often the clock is re-read. A refresh only costs work if a target actually moved.
`visualStepsPerTransition`	`28`	
`debugVisualStepsPerTransition`	`4`	
SEASON SHAPE		
`autumnLeafFallCompleteFraction`	`0.68`	
`springLeafReturnCompleteFraction`	`0.6`	
`autumnFlowerFallCompleteFraction`	`0.78`	
`mushroomsAutumnOnly`	`true`	
`minimumWinterLeafRetention`	`0.0`	Floor on winter leaf retention. 0.0 is canonical: winter is bare.
SNOW AND ICE		
`seasonalSnow`	`true`	Master switch for physical seasonal snow.
`seasonalWaterFreezing`	`false`	The pluggable lake-ice effect. Off by default; turning it on changes an existing world.
`maximumPhysicalSnowCoverage`	`1.0`	Caps the footprint. 1.0 is the canonical value.
`plantSnowOverlay`	`true`	
`migrateLegacy022Snow`	`true`	One-time adoption of snow left by a much older version.

OPTION	DEFAULT	NOTES
`winterSnowReplacesWildFlora`	`true`	Whether arriving snow buries wild flora rather than sitting beside it.
`maximumAccumulatedSnowLayers`	`8`	Deepest snow, in eighths. 8 is canonical.
SERVER BUDGET		
`surfaceColumnsPerTick`	`1024`	Surface columns per tick. Clamped hard outside a timelapse.
`leafColumnsPerTick`	`128`	Canopy columns per tick - the most expensive pass.
`debugLeafColumnsPerTick`	`512`	
`columnsPerChunkSlice`	`16`	
`maxVisibleSnowWritesPerTick`	`128`	Block writes per tick for snow.
`maxVisibleLeafWritesPerTick`	`24`	Block writes per tick for leaves.
`debugMaxVisibleLeafWritesPerTick`	`96`	
`maxVisibleLeafWritesPerChunkSlice`	`48`	
`debugImmediateLeafPriorityRadiusChunks`	`6`	
`canopyScanDepth`	`128`	How far below the treetop the canopy sweep looks.
`materializationSteps`	`24`	
`backgroundReconcileRadiusChunks`	`14`	Radius kept converged in the background.
`playerPriorityRadiusChunks`	`12`	Radius given priority around each player.
`preSendSeasonProjection`	`true`	Canonicalise a chunk before it is sent. Off, a chunk can arrive carrying a stale season.
DYNAMIC TREES		
`dynamicTreesSeasonManager`	`true`	
`physicalDeciduousLeafFall`	`true`	Whether leaves are physically removed rather than only tinted.
`protectDormantDynamicTreeBranches`	`true`	
`suppressDormantDynamicTreeLeafSpread`	`true`	
WEATHER		
`customWeatherSystem`	`true`	Season-aware weather instead of vanilla's.
`historicalWeather`	`true`	
`weatherEvaluationTicks`	`100`	
`weatherPatternMinutes`	`20`	
`debugWeatherPatternSeconds`	`12`	
CLIENT RENDERING		

OPTION	DEFAULT	NOTES
`nearSeasonalTinting`	`true`	Client-side seasonal tint on near blocks.
`nearVegetationFlattening`	`true`	
`maximumGroundPlantFlattening`	`0.14`	
`nearRefreshRadiusChunks`	`14`	
`nearRefreshChunksPerTick`	`2`	
`debugNearRefreshChunksPerTick`	`2`	
VOXY		
`voxySeasonRendering`	`true`	Master switch for the whole Voxy integration.
`voxySnowBlendStartBlocks`	`96.0`	Distance at which the LOD snow blend begins.
`voxySnowBlendEndBlocks`	`384.0`	Distance at which it is fully applied.
`suppressSeasonalVoxyReingest`	`true`	Keeps seasonal writes out of the LOD store. Off, the store carries seasons and goes stale.
`voxyServerAutoBackfillExistingRegions`	`true`	Ask VoxyServer to build LOD for regions that have none.

Tuning for server load

If the season system is taking too much of a tick, the two dials that matter most are the time budgets: `worldReconcileBudgetMicros` for normal running and `debugReconcileBudgetMicros` for while a timelapse is active. Lowering them gives the season system less of each tick and slows convergence; nothing breaks, the world simply catches up more gradually.

The column and write counts are secondary — they cap how much is attempted, but the time budget usually binds first. `canopyScanDepth` is worth lowering on worlds with very tall custom trees, since the canopy sweep reads one block state per level per column.

> **Timelapse arithmetic.** A 3600-second year is roughly nine thousand times real time. A full pass over a thousand loaded chunks is tens of seconds of budgeted work whatever the settings, so the far field will always trail a fast timelapse. It converges evenly rather than stalling; at real-calendar speed the question does not arise.

Part 8

Diagnostics and troubleshooting

Seasonal faults are hard to report and hard to reproduce: they appear kilometres away, in one season, after an hour of play. This part is about turning "it looks wrong" into a number.

Commands

COMMAND	WHAT IT DOES
`/teslesseasons status`	Current frame, phase, all channels, and the server status line.
`/teslesseasons season <season>`	Force a season.
`/teslesseasons phase <phase>`	Force a phase.
`/teslesseasons date <date>`	Force a calendar date.
`/teslesseasons transition <…>`	Drive a transition directly.
`/teslesseasons clear`	Drop every override and return to the real calendar.
`/teslesseasons timelapse <seconds>`	Run a whole year in that many seconds.
`/teslesseasons timelapse stop`	Stop it.
`/teslesseasons weather <…>`	Weather control; forces clear during a timelapse.
`/teslesseasons reconcile`	Force a reconcile pass now.
`/teslesseasons voxy backfill`	Ask VoxyServer to fill in regions with no LOD.
`/teslesseasons diagnostic capture`	One screenshot plus a state and world sample.
`/teslesseasons diagnostic year [seconds]`	A full year, captured automatically. Default 600.

The diagnostic bundle

`/teslesseasons diagnostic year` runs a whole simulated year and writes a ZIP under `teslesseasons-diagnostics/`. Send that ZIP when reporting a visual problem: it turns a claim about what the world looks like into something checkable.

FILE	CONTENTS
`timeline.csv`	Every season channel, once per second, for the whole year.
`NN-<phase>-state.txt`	All frame channels at that checkpoint, plus Voxy counters, player position, heap, blend distances.
`NN-<phase>-world-sample.csv`	A fixed grid of 2401 columns: surface block, category, snow layers, the block below, whether it is slab terrain.
`NN-<phase>.png`	The main render target, so Iris, Sodium and Voxy are all included.
`server-state.txt`	The server status line at capture time.
`environment/`	The relevant configs, as actually loaded.
`mods.txt, latest-log-tail.txt`	Installed versions and the log.

Checkpoints key on season channels, not wall-clock time, so each lands at the same point of the year at any timelapse speed. The world sample uses the same 2401 columns at every checkpoint after the first, which makes it a controlled experiment: the same coordinate can be followed through the whole year.

Reading the status line

```
loaded=1325 rr=1325 urgent=1185
surfaceCols=9474959 leafCols=8091919 snow+=1662490 snow-=257616 layers=1300818
leaves-=4281976 leaves+=96136 flora-=2028609 flora+=143601 preSendQueued=16657
ledger[leaves=…,flora=…]
voxyNeutral[snapshots=…,fast=…,copiedSections=…,snowStripped=…,leavesRestored=…,failures=…]
```

FIELD	MEANING, AND WHAT A BAD VALUE LOOKS LIKE
`loaded / rr / urgent`	Chunks tracked, in the round-robin, and queued urgently. Urgent staying near `loaded` means the queue is not draining.
`snow+ / snow− / layers`	Placements, removals, and depth changes. `layers=0` during Winter Stable would mean snow is never deepening.
`leaves− / leaves+`	Removed and restored. Over a full year these should approach each other; a large imbalance means the ledger is not surviving.
`ledger[…]`	Entries still owed back across loaded chunks. This is the only record of which leaf stood where. Empty while the world is bare means nothing can restore the canopy.
`voxyNeutral fast=`	Chunks handed to Voxy untouched because their ledgers were empty. **A bare winter chunk on that path is the failure** — the record of its canopy was gone when Voxy asked, so the LOD it wrote is winter permanently.
`voxyNeutral failures=`	Non-zero means neutralisation is throwing. Nothing downstream can be trusted.

The client state file carries the shader side:

```
voxyShader: vertexPatched=true fragmentPatched=true status=OK
            uniformsMax=12/12 seasonPrograms=1/7 lastBindAgeMs=31
```

`status` is the summary. `BRIDGE-NOT-APPLIED` means the shader injection did not match Voxy's source. `NO-PROGRAM-CARRIES-UNIFORMS` means it applied but no program resolved any season uniform. `PARTIAL(n/12)` means some resolved and some did not.

Symptom index

WHAT YOU SEE	WHERE TO LOOK
Distant trees stay bare after winter	The LOD store was written while that region was bare. The projector removes leaves; it cannot grow ones the store does not contain. Check `voxyNeutral fast=` and the ledger size, then rebuild the store (below).
A band of the wrong season at one distance	Same cause. The band is where you were standing when that region's LOD was written.
Only the area around the player updates	Convergence, not correctness. The player hotset is unbudgeted; everything else is a budgeted sweep. At a fast timelapse the far field cannot keep up. Check `urgent` against `loaded`.
Snow in the treetops, or shadows under bare trees	A surface search that stopped at the canopy. Fixed; if it reappears, the LOD in view predates the fix — rebuild.
Lakes are flat white or pale sheets	A shader snow cap reaching water. Fixed; check `fragmentPatched` and that the client jar is current.
Ground you can see through, in bands	Snow removal at coarse LOD replacing a voxel with air where there was nothing below. Fixed; the bands fall on section boundaries, every 32 blocks of a level.
Holes punched in the snowfield during the thaw	Melting without the per-layer feather. Fixed; a column now steps 4/8, 3/8, 2/8, 1/8, ground.
Bare patches on slab terrain in spring	The ground-cover placeholder could not survive on a slab and fell through to air. Fixed by an ordered candidate list.
A whole category never changes	Check the classification log line at startup. A category showing a very low count is a misclassification — that is how eight of nine berry blocks were found sitting in the wrong bucket.
Server tick time high	Lower `worldReconcileBudgetMicros` and `debugReconcileBudgetMicros`. Confirm the season system is the cause first: for most of the year its column passes short-circuit and cost almost nothing.

Rebuilding the LOD store

> **When a rebuild is the answer.** Anything of the form "a region shows the season it was in when I was last there" is stale LOD, and no amount of remeshing fixes it — the data Voxy holds is simply the wrong world. A rebuild from region files re-derives the LOD through the neutraliser, which applies each chunk's persisted ledgers as it goes, so the result is neutral regardless of what season the files were saved in.

Use VoxyServer's import to rebuild from the region files on disk. Watch the log for:

```
TESLES neutralising imported Voxy LOD: N seasonal snow removed,
                                       N leaves restored, N flora restored.
```

If that line never appears, the import ran without neutralisation and the store will hold whatever season the region files were saved in. That is a bug worth reporting rather than working around.

> **Server-side.** Everything that decides what the LOD store contains runs on the server: the neutralisation of a chunk before Voxy sees it, and the rule about which seasonal writes may reach the LOD at all. A client-only update changes none of it.

Part 9

Verification

What the build proves before it produces a jar. 93 tests across 17 suites, plus two verifiers that exist because both failure modes have actually shipped here.

The suites

SUITE	TESTS	WHAT IT PROVES
SeasonCheckpointContractTest	2	96 phase checkpoints — 12 phases × 8 progress values — match the canonical contracts, plus the named examples from the specification.
SeasonContinuityTest	3	168 boundary checks: 14 continuous channels × 12 phase boundaries, exact match.
SeasonFieldStatisticsTest	5	The coordinate field is uniform, pure, and its per-channel salts are independent.
VoxySnowParityTest	13	Physical, LOD-mesh and shader snow targets agree with zero mismatches; targets stay identical for a whole revision; the melt keeps its footprint and never drops a column from 2/8 or deeper straight to bare ground.
VoxyShaderPipelineTest	9	The real GLSL injection chain, run against Voxy's actual shaders: anchors match, every function is defined once, every uniform is declared, and patching is idempotent.
SeasonNeutralityTest	6	Only writes that heal the world toward neutral reach the LOD store, so a store built in any season converges on the neutral one.
NeutralPersistenceTest	4	Seasonal changes are recognised by the same field the server decided with, so they never reach the LOD database.
StaleSeasonEliminationTest	5	No path leaves an older season visible.
SeasonRevisionChurnTest	8	Bounds how often the world is told to redo itself, sampled at the real 30-second clock rate.
ColumnSweepTest	3	A chunk's column sweep is a true permutation, so no column can starve however fast the calendar moves.
FloraClassificationTest	8	Classification against the real Minecraft block registry, and everything that must never be touched.
WildBerryClassificationTest	4	Wild berry bushes reach the berry channel whatever block class they extend; cultivated crops never do.
FloraChannelTest	4	Every flora category tracks its own frame channel and its own field salt.
SnowSupportTest	5	Snow rests on ground and never on water, ice or foliage — the same verdict on the server and in the LOD projector.
GroundCoverFallbackTest	4	Melting snow always has a placeholder to fall back on, including on slab terrain.

SUITE	TESTS	WHAT IT PROVES
TallPlantAtomicityTest	3	Both halves of a double-height plant always reach the same verdict.
ModularityTest	7	Seasons and world effects can be replaced, keep their order, are skipped when inactive, and cannot register without an id.

> **Real registries, real shaders.** Minecraft's bootstrap runs in the test JVM, so classification is checked against the actual block registry rather than a mock. The shader suite runs the real injection chain against Voxy 0.2.18-beta's actual shader source, so a Voxy update that moves an anchor fails the build rather than the game.

The two wiring verifiers

Both run as part of `build`, before the jar is produced, and both exist because the failure they catch has actually shipped.

TASK	WHAT IT CHECKS
`verifyModWiring`	Every mixin is registered in a config, every entrypoint resolves, no `ClientModInitializer` is orphaned. Version 0.7.0 carried fifteen mixins that no config referenced and an initializer that was not an entrypoint — the entire Voxy seasonal projection, silently inert, invisible by reading.
`verifyMixinTargets`	Every `@Mixin` target class and injected method descriptor resolves against real bytecode, and no mixin targets the mod's own code. The same version carried mixins aimed at methods that had been renamed away, which would have killed the game at startup had they been registered.

The second half of that check — no mixin targets our own code — is a design rule, not a safety one. If behaviour belongs in a class, it lives in that class. Runtime self-patching hides logic from readers and from tests.

What the tests do not cover

Honesty about the boundary is more useful than a claim of completeness.

- **Rendering.** No test looks at a pixel. The shader suite proves the GLSL is well-formed, injected once and declares its uniforms; it cannot prove the result looks right.
- **Live server behaviour.** Budgets, convergence rate and tick cost are only observable in a running world. The diagnostic bundle exists for exactly this gap.
- **Third-party updates.** The mixin targets are verified against the exact Voxy, VoxyServer and Dynamic Trees builds in `libs/`. A different build may invalidate a target — which the verifier will catch at build time, not at runtime.

Part 10

Building from source

```
cd TeslesSeasons
./gradlew build
```

The jar lands in `build/libs/teslesseasons-1.0.0.jar`. `build` runs the contract suites and both wiring verifiers first, so a build that succeeds has already checked everything in Part 9.

COMPONENT	VERSION	NOTES
JDK	25	Minecraft 26.2 requires it.
Gradle	9.5.1	Provided by the wrapper.
Fabric Loom	1.17.19	Resolved automatically.
Minecraft	26.2	Downloaded automatically.

If the wrapper picks the wrong JDK:

```
JAVA_HOME=/path/to/jdk-25 ./gradlew build
```

Why there is an identity-mappings jar

Minecraft 26.2 ships **deobfuscated**. Mojang publishes no mappings artifact for it, and Fabric's intermediary for 26.2 is the empty `0.0.0` set. Loom still requires a mappings dependency, so `libs/identity-mappings-26.2.jar` supplies a tiny-v2 file with an `official → named` header and no entries: every name maps to itself. It is generated from `build-support/identity-mappings/` and is why `loom.useIntermediateMappings = false` is set.

Integration jars

`libs/` holds the exact third-party builds this mod is compiled against — Voxy 0.2.18-beta, VoxyServer 1.2.4, Dynamic Trees, TeslesWorldGeneration. They are `compileOnly`. Every mixin target was verified against these exact binaries; replacing them with different builds may invalidate a target, which `verifyMixinTargets` will report.

Appendix A

Mixin inventory

Every mixin, what it attaches to and why. All are `@Pseudo` where they target another mod, so a missing mod is not a crash.

MIXIN	SIDE	PURPOSE
VOXY — MESH AND SHADER (CLIENT)		
VoxySeasonMeshProjectionMixin	client	Projects the frame onto LOD geometry at mesh-build time. The core of Part 5.
VoxySeasonGeometryCacheBypassMixin	client	Refuses a cached mesh built under an older geometry key.
VoxyWatchedSectionMixin	client	Tracks which sections are live, for the remesh sweep.
VoxyShaderLoaderMixin	client	Injects the three GLSL layers as shaders load.
VoxyShaderUniformMixin	client	The one and only binder for the twelve season uniforms.
VoxyModelFactoryCategoryMixin	client	Attaches a seasonal category to each LOD model id.
VoxySeasonAwareModelDedupMixin	client	Keeps deduplication from collapsing models that differ only by season category.
VoxyClientLevelDirtyMixin	client	Marks client-side sections dirty on seasonal change.
VOXY — NEUTRAL PERSISTENCE		
VoxySnapshotIngestFixMixin	both	Hands Voxy a neutralised copy of a chunk instead of the live one.
VoxyImporterSnowNeutralizerFixMixin	both	Strips seasonal snow during import.
VoxyWorldImporterNeutralizerMixin	both	Applies the persisted ledgers to imported region sections.
VoxyServerDirtyTrackerMixin	both	Holds back LOD updates for writes that diverge from neutral — and lets healing writes through.
VoxyRawIngestSuppressionMixin	client	Blocks ingest of a recognised seasonal mutation.
VoxyRemoteAuthorityGuardMixin	client	Disables local ingest while a server is supplying LOD.
VoxyClientStorageEpochMixin / VoxyServerStorageEpochMixin	both	Cache-epoch handling across versions.
VoxyServerInstanceMixin	both	Hooks VoxyServer's lifecycle.
WORLD AND COMPATIBILITY		
PlayerChunkSenderMixin	both	Canonicalises a chunk before it is sent to a player.
GrassSnowIntegrityMixin	both	Keeps the snowy grass property consistent with what is above it.
DynamicTreesLeafTickMixin	both	Stops dormant trees regrowing leaves the season removed.

MIXIN	SIDE	PURPOSE
DynamicTreesBranchRotGuardMixin	both	Stops branches rotting because winter took their leaves.
WEATHER (CLIENT)		
CustomWeatherRainStrengthMixin	client	Season-aware precipitation strength.
WinterWeatherRendererMixin / WinterWeatherColumnMixin	client	Snow rather than rain in winter, per column.

Appendix B

Glossary

TERM	MEANING IN THIS MOD
Frame	An immutable `SeasonFrame`: what the world should look like now, as absolute targets.
Channel	One number in a frame — `leafRetention`, `snowDepth` and so on.
Target	A channel's value, quantised to 1/256 at the point of selection.
Revision	Monotonic counter, bumped only when targets change. A coalescing token.
Sector	A season module: a pure function from a clock snapshot to a frame.
Coordinate field	The pure hash turning `(position, seed, salt)` into [0, 1), which decides membership.
Salt	A per-channel constant that keeps channels from correlating.
Ledger	Per-chunk persistent record of what the season removed or placed. What makes seasonal change reversible.
Ownership	Whether a block is in a ledger. Only owned blocks are ever removed.
Neutral world	Full canopy, full flora, no seasonal snow. What Voxy's LOD store is meant to hold.
Divergent / healing write	A change away from the neutral world, or back toward it. Decides whether it may reach the LOD store.
Geometry key	Identity of everything that changes LOD geometry. Decides whether a cached mesh may be reused.
Projection	Applying the frame to something — blocks, LOD geometry, or shader colour.
Sweep	One full pass over a chunk's 256 columns, in a permuted order.
Handoff	The band where real blocks give way to LOD.
Effect	A pluggable `SeasonalWorldEffect` — a seasonal world behaviour that is not part of the pinned contract.